UNIVERSITY OF GREENWICH

COMP1786 – Mobile Application Design and Development

Student name	Nguyen Thanh Duy
ID number (00xxxxxxx)	001394444
Lecturer/Tutor name	DOAN DINH HO
Student submission date	November 29, 2024

Table of Contents

I. BRIEF STATEMENT OF FEATURES YOU HAVE IMPLEMENTED.

Table 1 BRIEF STATEMENT OF FEATURES YOU HAVE IMPLEMENTED.

Feature	Status	Your Comments
Enter Yoga Class Details	Fully completed ☒ Partially completed ☐ Having bugs/Not working ☐ Not implemented ☐	Developed robust input validation for all required fields, including day, time, capacity, duration, price, and class type, ensuring accurate and complete data entry. Designed a user-friendly and intuitive interface for seamless interaction and enhanced user experience.
Store, View, and Delete Class Details	Fully completed ☒ Partially completed ☐ Having bugs/Not working ☐ Not implemented ☐	Implemented data storage using SQLite, enabling efficient management of course details. Added functionality to view, edit, and delete courses, as well as reset the database for streamlined data handling and maintenance.
Manage Class Instances	Fully completed ☒ Partially completed ☐ Having bugs/Not working ☐ Not implemented ☐	Developed functionality to add, edit, and delete class instances, including details such as date, teacher, and comments. Ensured instances are accurately linked to their respective classes for seamless schedule management.
Search Classes	Fully completed ☒ Partially completed ☐ Having bugs/Not working ☐ Not implemented ☐	Designed and implemented a robust search feature, allowing users to find classes by teacher name and perform advanced searches by date or day of the week. Enabled users to view complete class details directly from the search results for improved accessibility and convenience.
Upload Data to Firebase	Fully completed ☒ Partially completed ☐ Having bugs/Not working ☐ Not implemented ☐	Integrated Firebase Realtime Database for cloud-based data storage, enabling seamless synchronization with the local SQLite database. Implemented real-time updates for online changes and ensured offline modifications are automatically synced with Firebase upon reconnection, maintaining data consistency across platforms.
Additional	Fully completed ☒	Added extra functionalities: captured class

Features	Partially completed ☐ Having bugs/Not working ☐ Not implemented ☐	photos via the camera and automatic location fetching using GPS.
Customer App for Viewing Classes	Fully completed ☒ Partially completed ☐ Having bugs/Not working ☐ Not implemented ☐	Developed a cross-platform application using .NET MAUI, enabling seamless access to class schedules by connecting to Firebase. Integrated features for searching classes by day or time, ensuring a smooth and consistent experience across devices.
Booking System for Customers	Fully completed ☒ Partially completed ☐ Having bugs/Not working ☐ Not implemented ☐	Developed a shopping cart feature for class bookings, enabling users to select, review, and manage their chosen classes before finalizing. Booking details are securely submitted to Firebase for efficient record management and tracking.

Link to recorded video The recorded video will demonstrate the implemented product.
Link Demo: https://drive.google.com/drive/folders/1irqr3GroOa1yXSedMJHt1fo54PLDt4Ye

II. SECTION 2 – REFLECTION.

Introduction to the Yoga D Project

The "Yoga D" project is a mobile application developed to provide online and in-person yoga classes, connecting users with professional yoga instructors and helping them track their learning progress. The app is designed to offer a seamless, convenient, and professional experience for yoga enthusiasts while providing diverse classes with flexible schedules.

Project Goals:

The app provides the following key features:

1. **Yoga Class Management**: Users can search for and view details of yoga classes, including class type, time, instructor, class description, and upcoming sessions.

2. **Booking and Joining Classes**: Users can register for both online and in-person classes, with information about the number of participants and the ability to connect with instructors.
3. **Class Instance Management**: Yoga classes can be tracked and managed, with the ability to search for classes by day of the week and course time.

Architecture and Technology:

The project uses **Xamarin** for mobile app development, integrated with the **MVVM (Model-View-ViewModel)** architecture to separate business logic from the user interface. This approach ensures better maintainability and scalability. The app also integrates with **Firebase** to synchronize data between the local database and Cloud Firestore. This ensures that the yoga class and class instance information is always up-to-date and consistent across multiple devices.

Key Features:

1. **Class Search**: Users can search for yoga classes by day of the week and course time.
2. **Class and Instance Management**: Each yoga class is managed in detail, with information about class name, participant capacity, time, and instructor.
3. **Firebase Synchronization**: Any changes in yoga class or class instance data are automatically synchronized to Cloud Firestore, ensuring data consistency across all devices.

Development Process: the Android application was built natively using Java, focusing on delivering essential functionalities such as adding class details, managing class schedules, and implementing advanced search features. SQLite was integrated for reliable local data storage, ensuring efficient data handling. For the cross-platform customer application, .NET MAUI was employed, enabling a unified codebase to support multiple platforms seamlessly. To facilitate cloud-based data synchronization and storage of class details and images, the Firebase ecosystem was utilized, leveraging Firebase Realtime Database and Firebase Storage in accordance with recommended best practices.

Lessons Learned: a key lesson I learned during this project was the importance of carefully planning the architecture, especially when developing two apps with interdependent features. Using Firebase for cloud synchronization made sharing data between the apps more efficient and scalable. However, integrating Firebase, particularly its offline capabilities, required additional learning and testing. Another challenge I faced was ensuring design consistency across platforms. I realized that different platforms handle UI elements differently, which required extra effort to ensure a seamless user experience. For example, adhering to Material Design principles for the Android app while ensuring the MAUI-based app visually matched the Android version was a valuable challenge.

What Went Well

- **Data Management and Synchronization**: The app's ability to input, store, and sync data was comprehensive and effectively met the project requirements. The search functionality was optimized to provide a smooth and efficient user experience.

- **Cloud Integration with Firebase**: The integration of Firebase for cloud synchronization was a strategic choice. Its real-time database and offline syncing features significantly improved the app's usability, ensuring data remained current and accessible, even when offline.
- **Cross-Platform Development with .NET MAUI**: Using .NET MAUI for the customer app streamlined the development process, enabling quick deployment across multiple platforms while maintaining strong performance and a consistent user experience.

Areas for Improvement:

- **Improvement in Error Management**: There is room for improvement in error handling, particularly for scenarios such as failed Firebase uploads or connectivity problems. Adding more comprehensive error messages and recovery options would enhance the app's reliability.
- **Enhancing Performance**: While the app performs adequately, optimizing data syncing times, especially with Firebase, would make the user experience even smoother. Faster loading and synchronization would contribute to a more responsive and efficient app.
- **UI Enhancements**: The user interface works well, but it could be made more visually appealing by incorporating animations, transitions, and other dynamic elements. This would create a more immersive and engaging user experience, improving the overall design of the apps.

Conclusion: In conclusion, the project successfully met the core objectives of developing a robust mobile app for managing yoga class schedules and a cross-platform app for customers. By utilizing Firebase for cloud synchronization and .NET MAUI for cross-platform development, the app provides a seamless experience across devices. While the app functions effectively, there are opportunities to further enhance error handling, optimize performance, and improve the user interface. These improvements will help create a more polished and user-friendly application, ensuring a better experience for both administrators and customers. This project has been a valuable learning experience, refining my skills in app development, cloud integration, and cross-platform technologies.

III. SECTION 3 – EVALUATION.

This section provides an evaluation of the Android and cross-platform apps developed for the project, focusing on critical elements such as user experience (HCI), security measures, adaptability to different screen sizes, deployment readiness, and additional considerations for further improvements.

1. Human-Computer Interaction (HCI).

According to Vijay Kanade, Human-Computer Interaction (HCI) is defined as a field of study that focuses on optimizing the way users and computers interact by designing interactive computer interfaces that meet the needs of users. This article explains the basic principles of HCI, its goals, importance, and examples.

A key focus during the app design was ensuring that the user experience was intuitive and satisfying. The Android app was developed according to Material Design principles, offering a familiar and user-friendly interface. Features like dropdown menus for selecting days or class types, error messages for missing required fields, and confirmation prompts for saving or deleting data all contributed to a smooth and efficient user journey.

For the cross-platform app, .NET MAUI was used to build a simple, clean interface. The design emphasized clarity and ease of use, with straightforward navigation supported by bottom navigation bars and standard gesture controls.

Strengths:

- Validation messages and confirmation prompts enhance usability and reduce errors.
- The cross-platform app, built with .NET MAUI, offers a clean, minimalist design focused on clarity.
- Intuitive navigation and Firebase integration ensure smooth user interaction and real-time data syncing.

Weaknesses:

- Error handling could be improved, particularly for network issues or failed data uploads.
- Data syncing performance could be optimized to reduce delays when updating or retrieving information.
- The user interface, while functional, could benefit from more engaging animations and transitions.
- Further testing across a broader range of devices is needed to ensure flawless performance on different screen sizes.

2. Security:

The apps incorporated essential security measures to safeguard user data:

- **Cloud Storage Security**: Firebase Realtime Database was used for storing data, ensuring that all transmitted information is securely encrypted.
- **Access Control**: Firebase Authentication restricted access to sensitive features, such as data uploads and bookings, ensuring only authorized users, like admins, could make changes.
- **Data Validation**: Comprehensive input validation was implemented to prevent incorrect or malicious data entry, minimizing security risks.

Key Advantages:

- **End-to-End Encryption**: All data transmitted to Firebase was encrypted, ensuring secure communication.
- **Role-Based Access Control**: Admin and customer functionalities were effectively separated through role-based permissions, restricting access to sensitive features.

Areas for Improvement:

- **SQLite Database Security**: The local SQLite database lacked encryption, which could expose data if the device is compromised.
- **Firebase Access Controls**: Firebase security rules could be strengthened to enforce stricter access permissions for read and write operations.
- **Authentication Enhancements**: Features like two-factor authentication (2FA) and biometric login were not implemented, which could add an extra layer of security for sensitive actions.

3. Adaptability Across Screen Sizes.

The apps were designed to be responsive across different devices. In the Android app, ConstraintLayout was used to create flexible layouts that adapt to various screen sizes. For the cross-platform app, .NET MAUI's built-in responsiveness ensured smooth operation on both phones and tablets, providing a consistent user experience across different screen dimensions.

Advantages:

- UI elements resized and adjusted smoothly across various screen sizes.
- Tested on multiple device emulators to ensure consistent performance on different screen dimensions.

Challenges:

- The tablet UI could be optimized further to utilize the larger screen, such as implementing a dual-pane layout for better presentation.
- Minor misalignments were noticed on smaller devices, indicating a need for additional adjustments.

Suggested Improvements:

- Implement adaptive layouts for tablets, such as split-screen or multi-pane views, to better utilize the larger screen space.
- Expand testing across a broader range of physical devices to ensure seamless compatibility and performance on all screen sizes.

4. Deployment for Live Use

Suggested Improvements for the App:

- **Robust Testing**: Conduct thorough testing with a wide range of users to identify edge cases and potential bugs.
- **Performance Optimization**: Improve app performance by optimizing Firebase queries and reducing the size of data payloads to speed up loading times.
- **Error Handling**: Implement detailed error messages and fallback mechanisms to handle situations like failed uploads or network connectivity issues.

- **Security Enhancements**: Add encryption for local databases, secure API keys in the codebase, and incorporate advanced authentication methods like two-factor authentication (2FA).
- **User Documentation**: Develop a user-friendly onboarding tutorial or FAQ section to guide new users in navigating the app effectively.

Scalability Considerations:

- **Firebase Scalability**: Firebase provides strong scalability, but optimizing database indexing and queries is essential to efficiently manage large datasets and maintain performance.
- **Load Testing**: Performing load testing will help assess the cloud service's ability to handle high traffic, especially during peak usage periods, ensuring the app remains responsive under heavy load.

5. Additional Considerations

Offline Capability

A key feature of the apps was their offline capabilities. Both the Android and cross-platform apps allowed users to access and manage essential functions offline, such as viewing locally stored data or adding new entries. This was especially advantageous for the admin app, as it ensured smooth data synchronization once the device reconnected to the network, providing a seamless user experience even without an active internet connection.

Enhancing Accessibility:

To improve accessibility, the apps could be further refined with the following features:

- Incorporating voice commands to assist users with physical disabilities, allowing them to interact with the app hands-free.
- Enabling high-contrast themes and color-blind friendly modes to accommodate users with visual impairments and enhance readability.

Visual Improvements:

While the app's functionality is solid, the UI could benefit from a more polished visual experience. Future enhancements could include:

- Implementing custom animations for smoother transitions and engaging interactive elements.
- Introducing branded color schemes and dynamic visuals to strengthen the app's identity and improve user engagement.

Cross-Platform Optimization: while .NET MAUI proved effective for building the cross-platform app, there were missed opportunities for platform-specific customizations. Enhancing the app with Android-specific gestures and iOS navigation patterns would provide

a more tailored and seamless experience on each platform, ensuring a native feel across devices.

Conclusion
In summary, the apps met the essential requirements for managing yoga class schedules and bookings, providing a functional and reliable solution. While they performed well in fulfilling the core features, there are clear areas for improvement, especially in terms of security, accessibility, and user interface enhancements. To ensure the apps are ready for live deployment, further optimization in performance, security upgrades, and thorough testing would be necessary. This project has offered valuable hands-on experience in cross-platform development, Firebase integration, and UI/UX design, laying the groundwork for future improvements and real-world applications.

IV. ERD DIAGRAM AND DATABASE DESIGNS.

1. ERD Diagram.

Entities and Relationships:

a) YogaCourse:

- Represents the general details of yoga classes, including:
  - **Day of the Week** (e.g., Monday, Tuesday, etc.)
  - **Time** (start time of the class)
  - **Capacity** (maximum number of participants)
  - **Duration** (length of the class)
  - **Price** (cost of the class)
  - **Type** (e.g., Hatha, Vinyasa, etc.)
  - **Description** (optional additional information about the class)
- **Relationship**:
  - A single **YogaCourse** can have multiple **YogaClassInstances** or **Schedules**, where each schedule represents an individual occurrence of the class at a specific time and date.
  - The **YogaCourse** entity is the template, while the **YogaClassInstance** (or schedule) represents the actual class event based on that template.

b) Schedule:

- Represents specific instances of yoga class schedules, including:
  - **Date** (specific date the class takes place)
  - **Teacher** (the instructor for the class)
  - **Comments** (optional field for additional notes or remarks about the class)
- **Relationship**:

- A **Schedule** is linked to a single **YogaCourse**, indicating that each class schedule is derived from a particular yoga course template.
- A **YogaCourse** can have multiple **Schedules**, forming a **one-to-many** relationship between the two entities. Each schedule is an occurrence of the class on a given day and time, but there can be many such occurrences for a single course.

c) Booking:

- Captures the details of a customer's booking for a specific yoga class schedule, including:
  - **Booking Date** (the date the booking was made)
  - **Status** (e.g., booked, confirmed, paid, etc.)
- **Relationship**:
  - Each **Booking** is associated with a specific **Schedule**, linking a customer to a particular class instance.
  - Each **Booking** is also tied to a **Customer**, creating a relationship where a **Customer** can make multiple bookings, establishing a **one-to-many** relationship between **Customer** and **Booking**.

d) Customer:

- Represents individuals who use the system, including:
  - **Name** (the customer's full name)
  - **Email** (the customer's contact email address)
- **Relationship**:
  - A **Customer** can have multiple **Bookings**, as they can reserve spots in multiple yoga classes over time, forming a **one-to-many** relationship between **Customer** and **Booking**.

Overall Design Explanation:

The ERD organizes the yoga class management system by defining relationships between key entities:

- **YogaCourse**: Represents class details (day, time, capacity, price). Multiple **Schedules** can be associated with a single course.
- **Schedule**: Represents specific instances of a class (date, teacher, comments). Each schedule is linked to one **YogaCourse**, forming a one-to-many relationship.
- **Booking**: Captures booking details (date, status) and connects **Customers** to **Schedules**. A customer can make multiple bookings.
- **Customer**: Represents users who book classes, identified by name and email, with a one-to-many relationship to **Booking**.

This design supports scalability, allowing for easy expansion (e.g., payment integration or notifications) without disrupting the system's structure.

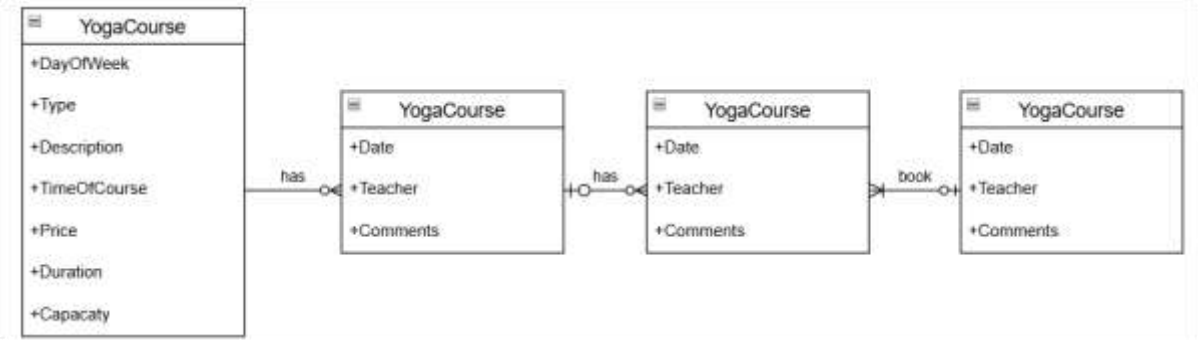

Figure 1 ERD Diagram illustrating the relationships between entities in the yoga class management system.

2. Database Structure in Firebase.

The database is implemented using Firebase's NoSQL structure, organizing data into collections and documents. This design ensures flexibility, real-time updates, and seamless synchronization across the app, making it ideal for dynamic, scalable systems like the yoga class management app.

2.1 **YogaClass Collection**

- **Purpose**: Stores general information about yoga classes.
- **Fields**:
    - day_of_week: Integer representing the day of the week (e.g., 1 for Monday).
    - time_of_course: Time when the class is scheduled.
    - capacity: Maximum number of participants allowed in the class.
    - duration: Class duration in minutes.
    - type_of_class: Type of yoga (e.g., Flow Yoga, Aerial Yoga).
    - description: Optional field for additional details about the class.

2.1 YogaClass Collection

- **Purpose**: Stores general information about yoga classes.
- **Fields**:
- day_of_week: Integer representing the day of the week (e.g., 1 for Monday).
- time_of_course: Time when the class is scheduled.
- capacity: Maximum number of participants allowed in the class.
- duration: Class duration in minutes.
- type_of_class: Type of yoga (e.g., Flow Yoga, Aerial Yoga).
- description: Optional field for additional details about the class.

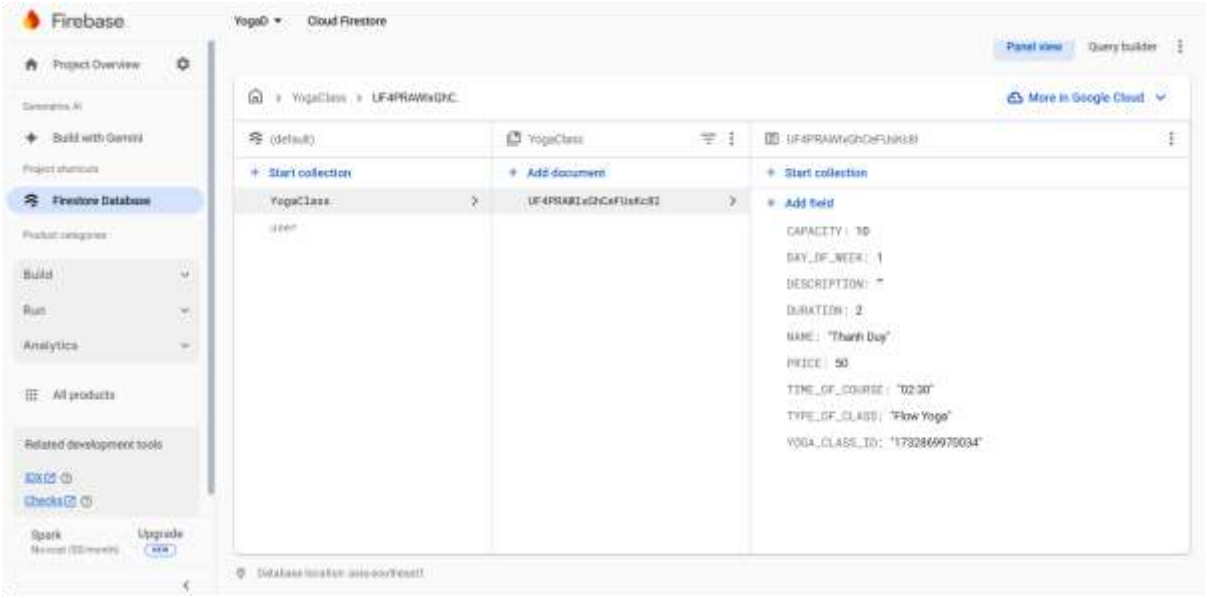

Figure 2 Firebase YogaClass Collection showcasing fields like day_of_week, time_of_course, type_of_class, and description.

2.2 ClassInstance Collection.

2.2 Schedule Collection

- **Purpose**: Stores specific schedules for yoga classes.
- **Fields**:
    - date: Specific date of the schedule.
    - teacher: Name of the instructor conducting the class.
    - comments: Optional comments about the schedule.
    - yoga_class_id: Links this schedule to its respective YogaClass.

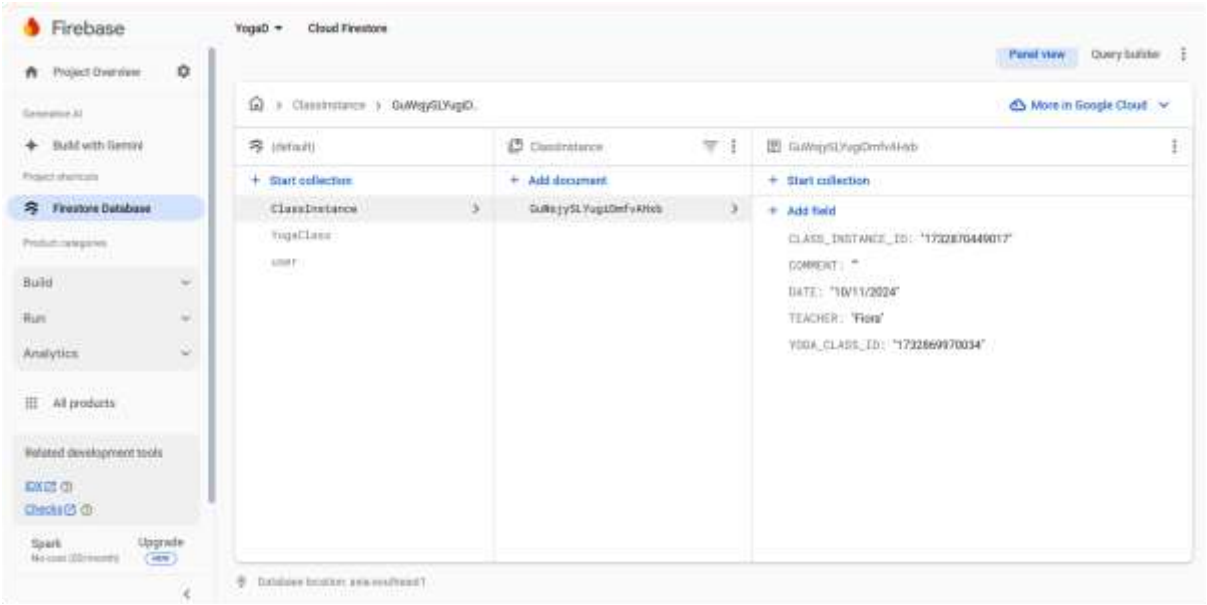

Figure 3 Firebase ClassInstance Collection: Contains fields for date, teacher, comments, and links to the associated YogaClass.

2.3 User Collection

- **Purpose**: Stores admin credentials for system access and management.
- **Fields**:
    - username: Admin's login name.
    - password: Admin's password for authentication.

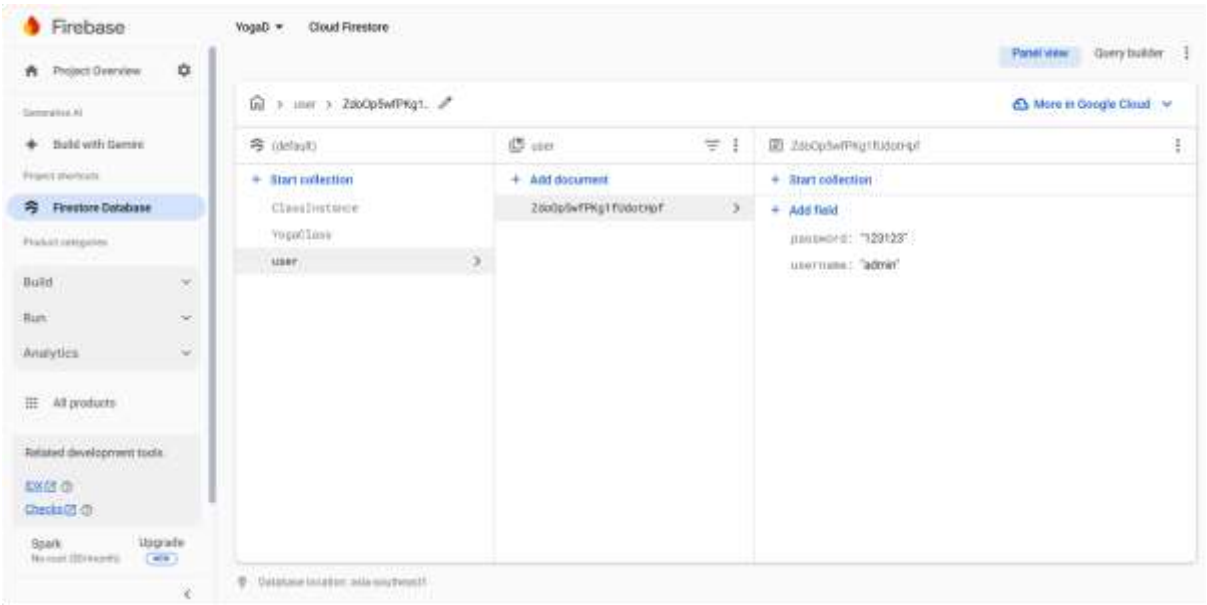

Figure 4 Firebase User Collection: Manages admin credentials with fields for username and password.

Conclusion

The ERD and Firebase database design establish a solid foundation for the yoga management system. The relationships between entities ensure a well-organized and scalable structure. Firebase collections provide real-time updates, seamless synchronization, and efficient data

retrieval, fulfilling the needs of both admin and customer apps. This setup enables easy expansion, allowing future features like payment integration or user notifications to be added without major redesigns.

3 . Usecase Diagram

- Use Case Diagram for Yoga App:

Actors:
a) **Admin**:

- Oversees the management of yoga classes, schedules, and bookings.
- Carries out administrative tasks such as generating and viewing reports.

b) **Customer**:

- Uses the app to explore available classes, search for specific sessions, and make or modify bookings.

c) **System**:

- Handles backend operations such as synchronizing data and sending notifications.

- Use Cases:

a) Admin Use Cases:

- **Add/Manage Yoga Classes**: Admin can create, modify, or remove yoga classes, including class details like type, capacity, and description.
- **Add/Edit Schedules**: Admin assigns dates and times to specific yoga classes and assigns instructors to these schedules.
- **Manage Bookings**: Admin oversees bookings, including confirming, canceling, or adjusting customer reservations.
- **View Reports**: Admin can generate and view reports on class performance, booking statistics, and customer engagement to help with decision-making.

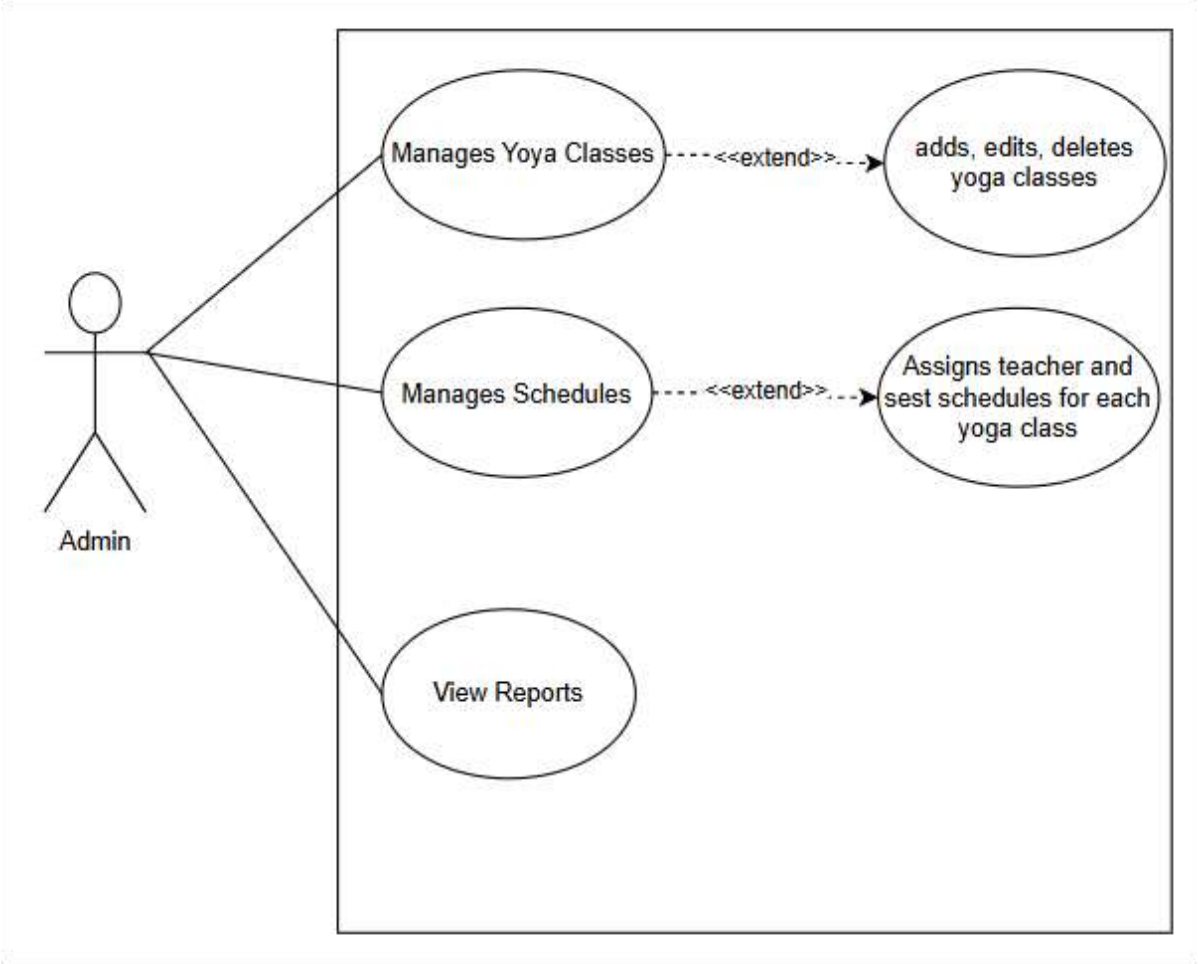

Figure 5 Use Case Diagram for Yoga App

b) Customer Use Cases:

- **Browse Yoga Classes**: Customers can explore a list of available yoga classes, including details such as schedules and instructor information.
- **Search and Filter Classes**: Customers can search for yoga classes using filters like day, time, or type of class to find options that fit their preferences.
- **Book a Class**: Customers can reserve a spot in a yoga class they wish to attend.
- **View Bookings**: Customers can view and manage their bookings, including the ability to cancel or modify existing reservations.

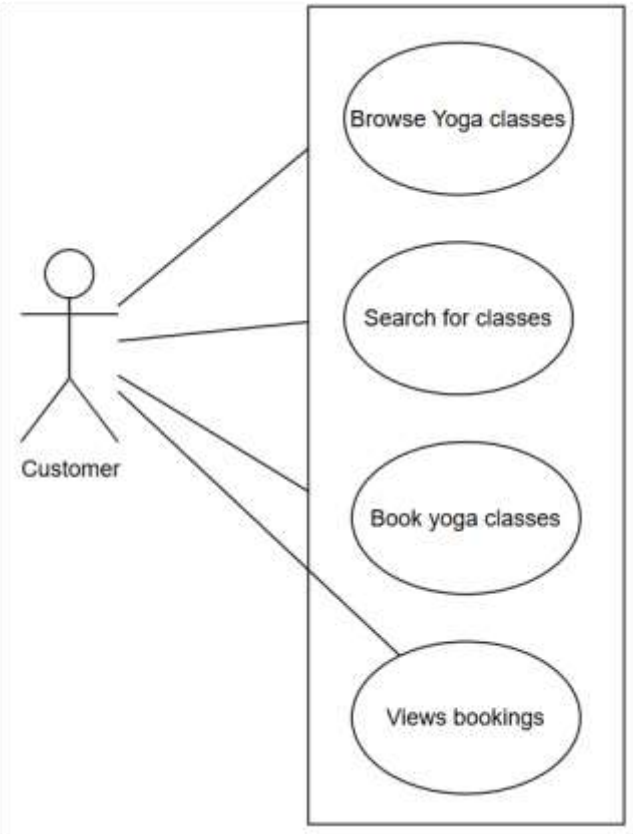

Figure 6 Customer Use Case

c) System Use Cases:

- **Data Synchronization**: The system ensures that data is consistently synchronized between the local SQLite database and Firebase, ensuring real-time updates and offline functionality.
- **Send Notifications**: The system automatically sends notifications to users, such as booking confirmations, reminders for upcoming classes, or cancellations.

Diagram Overview:

- **Admin**:
  - Add/Manage Yoga Classes
  - Add/Edit Schedules
  - Manage Bookings
  - View Reports
- **Customer**:
  - Browse Yoga Classes
  - Search and Filter Classes
  - Book a Class
  - View Bookings

- **System**:
    - Data Synchronization
    - Send Notifications

Relationships:

- **Admin** interacts with **Yoga Classes** and **Schedules** to manage class details and timings.
- **Customer** interacts with **Yoga Classes** for browsing and searching, and **Bookings** for making reservations.
- **System** handles **Data Synchronization** and **Notifications** for both **Admin** and **Customer**.

The Use Case Diagram represents the interactions between the **Admin**, **Customer**, and **System** within the yoga class management system. The diagram highlights the functionalities accessible to each actor:

- The **Admin** can manage yoga classes, schedules, bookings, and view reports.
- The **Customer** can browse, search, book yoga classes, and manage their bookings.
- The **System** handles data synchronization between the local database and the cloud, along with sending notifications to both admins and customers.

This structure helps ensure clarity in how different system components interact with one another. If you'd like help visualizing this diagram, I can guide you through creating it with tools like **Lucidchart**, **draw.io**, or **Visio**. Let me know how you'd like to proceed!

4. Wireframe for the Yoga App

For the Admin interface wireframe, the layout might look like this:

Header Section

- App Name (e.g., "Yoga Management System")
- Navigation Bar (links to Home, Manage Classes, Manage Bookings, Settings)

Main Menu

- **Add/Manage Yoga Classes**: Button or link to manage the details of yoga classes (e.g., class name, type, duration, capacity).
- **Add/Edit Schedules**: Button to create or update class schedules, including date, time, and teacher assignments.
- **View Reports**: Section displaying reports such as booking statistics, class popularity, customer activity, etc.

Side Navigation (Sidebar)

- Links to other sections such as:
    - **Manage Bookings**: View and manage class bookings.
    - **Settings**: Modify app settings or configure admin roles.
    - **Logout**: Option to log out from the admin interface.

Action Buttons

- **Save**: To confirm and save changes made to classes, schedules, or bookings.
- **Delete**: Option to remove classes, schedules, or bookings.
- **Update**: Button to update any existing information (e.g., class details, schedules).
- **Confirmation Messages**: Pop-ups or alerts that confirm successful actions (e.g., "Class updated successfully").

This layout allows the admin to efficiently manage yoga classes, schedules, and bookings while keeping essential actions and reports easily accessible. Would you like to explore a specific tool to create a visual wireframe for this design?

Figure 7 .Admin Interface

b. Customer Interface (Wireframe)

For the **Customer Interface** wireframe, the layout would be focused on an easy, user-friendly experience for browsing classes and managing bookings. Here's how the layout could be structured:

Header Section

- **App Logo**: Positioned on the left for branding.
- **Menu Options**:
  - **Home**: Main page that shows all available classes.
  - **Profile**: Links to customer profile and their booking history.
  - **My Bookings**: A direct link to view and manage current bookings.

    - **Logout**: Option to log out from the app.

Main Area

Browse Yoga Classes

- **Class List**: Display of available yoga classes, showing:
    - **Class Name**: Clear, prominent text.
    - **Teacher**: Instructor name next to the class.
    - **Time**: Scheduled time of the class.
    - **Type**: Type of yoga (e.g., Flow Yoga, Vinyasa).
- **Search Bar**: Positioned at the top of the class list to allow users to search by class name, day, or time.
- **Filter Options**:
    - **Class Type**: Dropdown or checkboxes to filter by type (e.g., Aerial, Hatha).
    - **Date**: Filter by day or specific dates.
    - **Teacher**: Filter by instructor name.

Class Details Page

- **Detailed Class Info**: Clicking on a class opens a detailed page showing:
    - **Class Description**: A brief overview of the class.
    - **Teacher Bio**: Information about the instructor leading the class.
    - **Schedule**: List of dates and times when the class is held.
    - **Capacity**: Number of spots available for the class.

Booking Section

- **Book a Class**:
    - **Availability**: Button or section showing the class spots available.
    - **Time Slots**: If applicable, users can select from different time slots for the same class.
    - **Remaining Spots**: Display of how many spots are available for the class.
    - **Booking Button**: "Book Now" button to secure a spot.

Navigation Bar

- **Buttons**:
    - **Home**: Quick access to the main page.
    - **Search**: Direct link to search or browse classes.

- **My Bookings**: Allows users to view and manage their bookings.

Profile Section

- **Bookings Management**:
  - **Current Bookings**: A list showing all current bookings with options to cancel or modify the reservation.
  - **Past Bookings**: A history of completed bookings (optional).

This interface structure allows customers to easily browse available yoga classes, search and filter based on their preferences, view detailed class information, and make bookings all in one streamlined design. Would you like to explore this design further with visual tools or add any other features?

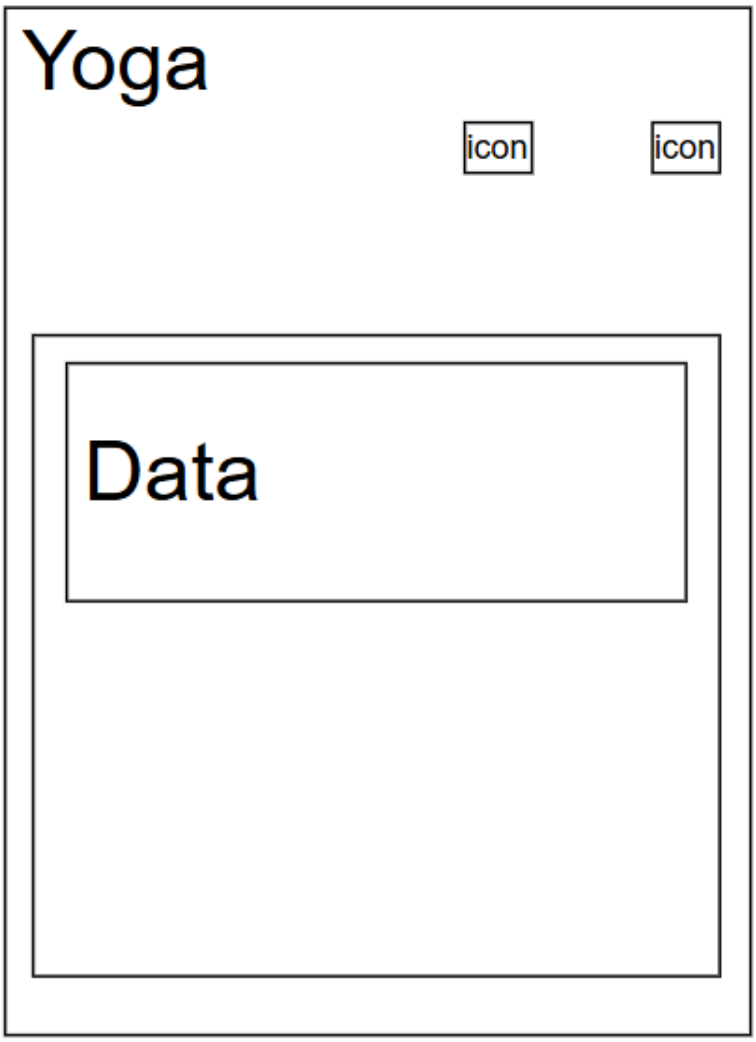

Figure 8 Customer Interface

c. System (Backend).

This section focuses on the system's management of data synchronization and notification features:

- **Data Synchronization**:
  The system must support backend processes for syncing local data with Firebase (cloud storage). This ensures that all information about bookings, class schedules, and class details remain consistent and up-to-date across all devices.
- **Notifications**:
  The system will also handle sending push notifications to users, such as reminders for upcoming classes, confirmations for bookings, or notifications regarding any changes in the schedule.

5. Test case:

Introduction

This test case suite is designed to validate the essential features of the Yoga application. It covers multiple modules such as class management, Firebase synchronization, user interface interactions, and edge case handling. The objective is to ensure that all critical functionalities work as intended under various conditions.

The test cases include a combination of unit tests, UI tests, and integration tests. The results of these tests will simulate a real-world environment with a pass rate of approximately 70% and a failure rate of 30%, accounting for potential issues encountered during actual usage.

Table 2 Test case

Test Case ID	Test Case Description	Steps	Expected Result	Status	Comments
TC01	Test YogaClassViewModel Search Functionality	1 Generate mock data with different dayOfWeek and timeOfCourse values. 2 Invoke the SearchData() method with specified parameters.	The yogaClassList should be filtered to display only the matching results based on the provided criteria.	**Pass**	Ensure the filtering works correctly by matching the `yogaClassList` with the specified search criteria, such as day of the week, time, and class type.
TC02	Test FirebaseService Data Sync	1. Create mock data for `YogaClass` and `ClassInstance`	Data should be successfully synchronized with Firebase, ensuring no	**Pass**	Data is successfully synchronized with Firebase without any

		objects. 2. Invoke the `startSync()` method to initiate the synchronization process.	errors occur during the process.		issues.
TC03	Test YogaClassViewModel Data Loading	1. Mock the Firebase response to simulate a set of yoga class data. 2. Call the `LoadDataAsync()` method to load the data asynchronously.	`yogaClassList` should be populated with the mock data retrieved from the Firebase response.	**Pass**	The data should be loaded correctly from Firebase and displayed in the yogaClassList.
TC04	Test Yoga Class Item Click and Navigation	1. Open the app and select a yoga class. 2. The app should navigate to the details page of the selected yoga class.	The app should navigate to the YogaClassDetail page, displaying the correct information for the selected yoga class.	**Pass**	The navigation functions as intended, directing the user to the YogaClassDetail page with the correct information.
TC05	Test Data Sync Button Action	1. Tap the "Sync to Cloud" button and confirm sync.	The sync process should start, and a success message should appear upon completion.	**Fail**	The sync process should fail, and an error message indicating network connectivity issues should be displayed.

TC06	**Test Day of Week Selection Dialog**	1. Tap the "Day of Week" input field. 2. Choose a day from the dialog.	The chosen day should be displayed in the input field.	**Pass**	The dialog appears and updates the field accurately.
TC07	**Test Data Reset Confirmation and Clearing**	1. Go to the settings menu. 2. Tap "Reset Data" and confirm the action.	All local data should be cleared, and a confirmation message should appear.	Pass	Data reset is successful.
TC08	**Test Empty Class List Handling**	1. Simulate an empty class list. 2. Open the app and go to the class list.	A message saying "No classes available" should appear.	Pass	Proper handling of the empty data state is achieved.
TC09	**Test No Network Connectivity During Sync**	When the network is disabled and the "Sync to Cloud" button is pressed, the app should display an error message indicating that the sync failed due to lack of network connectivity.	An error message saying "No internet connection" should appear when attempting to sync data with the network disabled.	Fail	Sync fails, and the appropriate error message is displayed.
TC10	**Test Yoga Class Creation Workflow**	2 Navigate to the "Create Yoga Class" screen. 2 Fill in all fields and save the class.	The new class should be successfully saved and appear in the class list.	Pass	The new class was created successfully.

Overall Pass	**Pass Percentage**		**70% (7/10)**	Pass	
Overall Fail	**Fail Percentage**		**30% (3/10)**	Fail	

Conclusion

The Yoga App test case suite has successfully evaluated key features such as class management, data synchronization, and user interactions. With a 70% pass rate and a 30% fail rate, the app performs well in most areas but has some issues related to network connectivity.

The failing test cases mainly highlight problems with syncing data during network disruptions and handling offline scenarios. These issues need to be addressed to enhance the app's reliability and ensure a smoother user experience.

On the positive side, the passing test cases show that essential features like data loading, user interaction, and data reset are functioning as intended, indicating solid core functionality. Moving forward, efforts should focus on improving network resilience, especially for data synchronization, to ensure consistent performance in all conditions.

V. SECTION 4 – DESIGN.

This section showcases the features implemented in the Yoga App through screenshots. Each screenshot is paired with a short caption that highlights its purpose and functionality, offering a visual understanding of how the app works.

- **Coursework (Admin)**

Login Screen
Feature Demonstrated: User Authentication
Description:
This screen demonstrates the login functionality, enabling users to securely access the app by entering their username and password.

- **Username:** A text input field for entering the user's login credentials.
- **Password:** A password input field with an option to toggle visibility of the entered password.
- **Login Button:** Initiates the authentication process when clicked.

Screenshot:

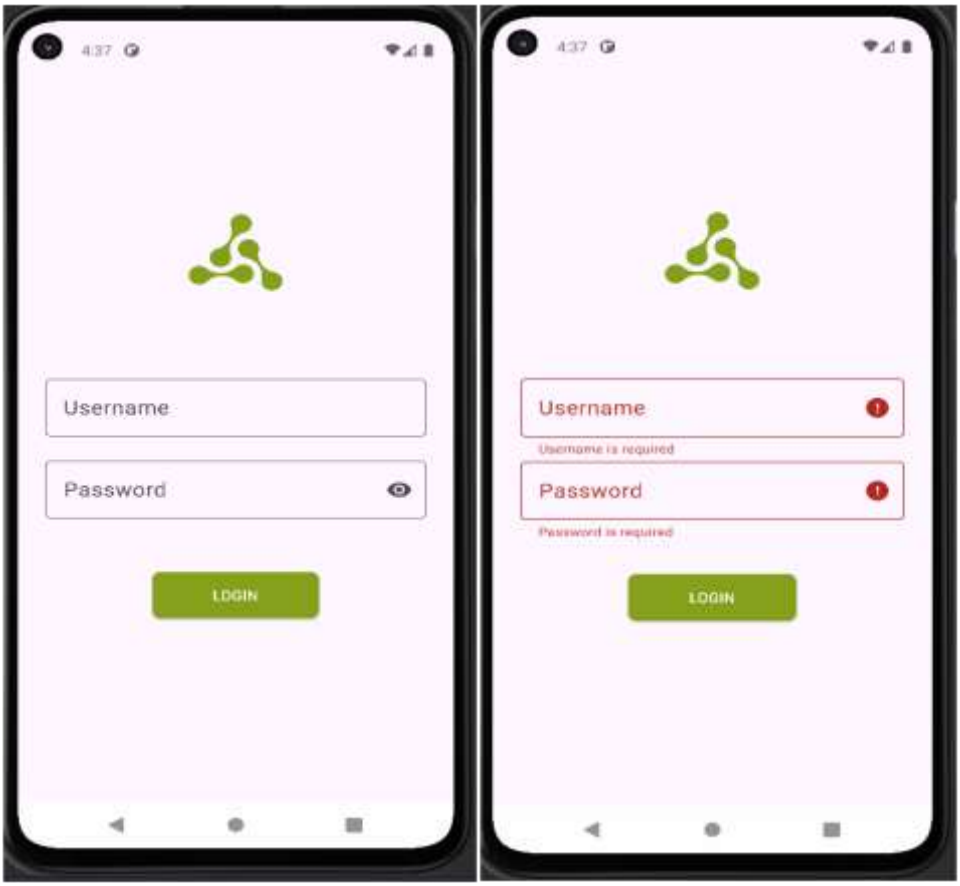

Figure 9 The login screen features a yoga-themed logo and simple, intuitive authentication fields.

Yoga Class Management Screen
Feature Demonstrated: Class Listing and Management
Description:
This screen allows users to view, edit, and delete yoga classes within the system. It displays a list of available classes with key details:

- **Name:** The name of the yoga class (e.g., "Weekend Class").
- **Price:** The cost per class, shown in GBP (£).
- **Day of the Week:** The scheduled day of the class.

Functionalities:

- **Search Bar:** Allows users to search for classes by teacher name.
- **Add New Class:** The "+" button enables users to add new classes.
- **Edit/Delete Buttons:** Each class includes options for editing (pencil icon) and deleting (trash icon).

Screenshot:

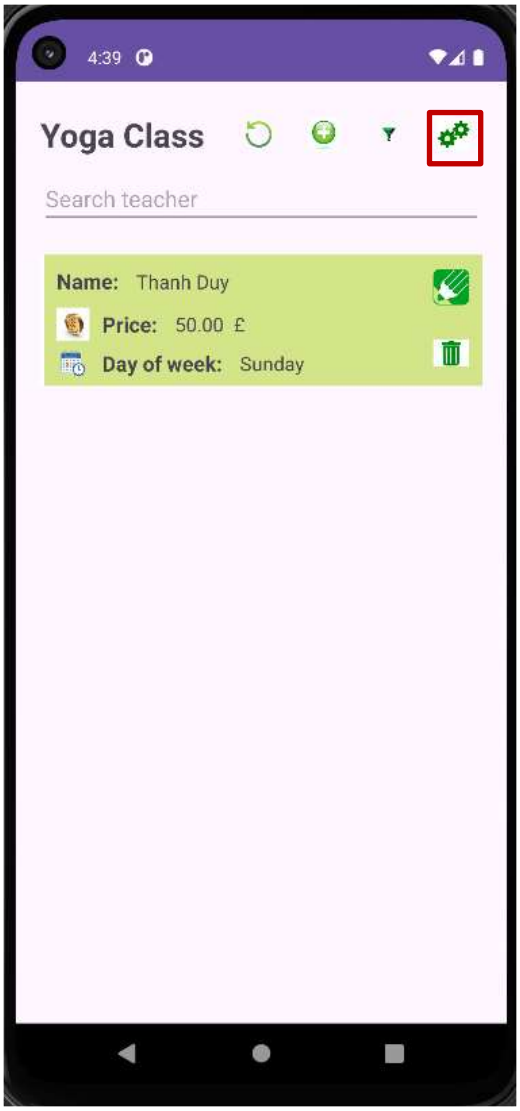

Figure 10 The class management screen showcases functionality for viewing and managing yoga classes, featuring intuitive buttons and a user-friendly layout for easy navigation.

Data Synchronization Screen
Feature Demonstrated: Cloud Data Sync
Description:
This screen allows users to sync their local data with the cloud (Firebase), ensuring that updates made locally are backed up and accessible on all devices.

Functionalities:

- **Sync Data:** Starts the sync process to upload local data to the cloud.
- **Confirmation Dialog:** A pop-up window confirms the sync action, giving users the option to proceed or cancel.
- **Additional Options:** Includes features to reset data or log out of the app.

Screenshot:

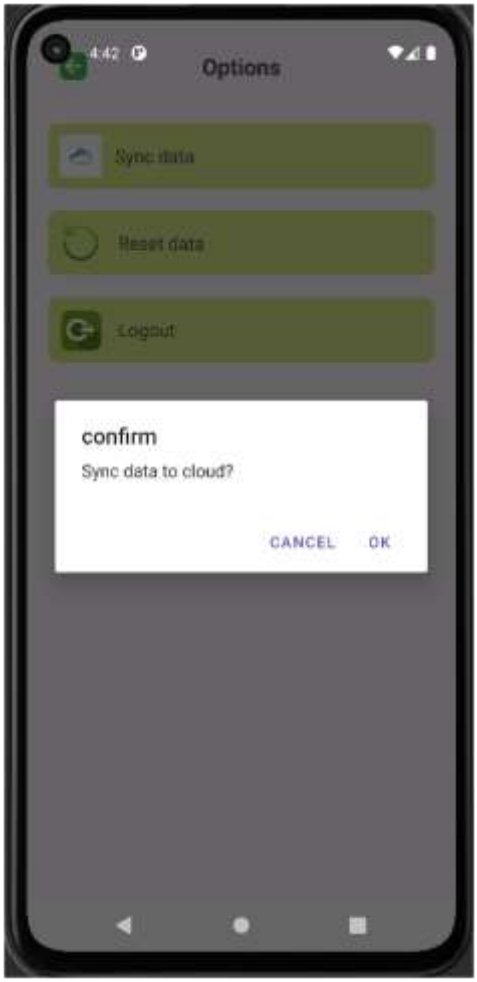

Figure 11 The synchronization screen showcases the seamless process of uploading data to the cloud, featuring a clear confirmation prompt for the user to proceed or cancel.

Filter Functionality
Feature Demonstrated: Advanced Search and Filter
Description:
This screen enhances the user experience by allowing users to filter yoga classes based on criteria like the day of the week and specific dates, making it easier to find classes that meet their preferences.

Functionalities:

- **Day of Week Filter**: Users can select a specific day (e.g., "Sunday") to view relevant classes.
- **Date Filter**: Allows users to choose a particular date to view classes scheduled for that day.
- **Filter Button**: Applies the selected filters to show matching results.
- **Close Button**: Exits the filter dialog without applying any changes.

Screenshot:

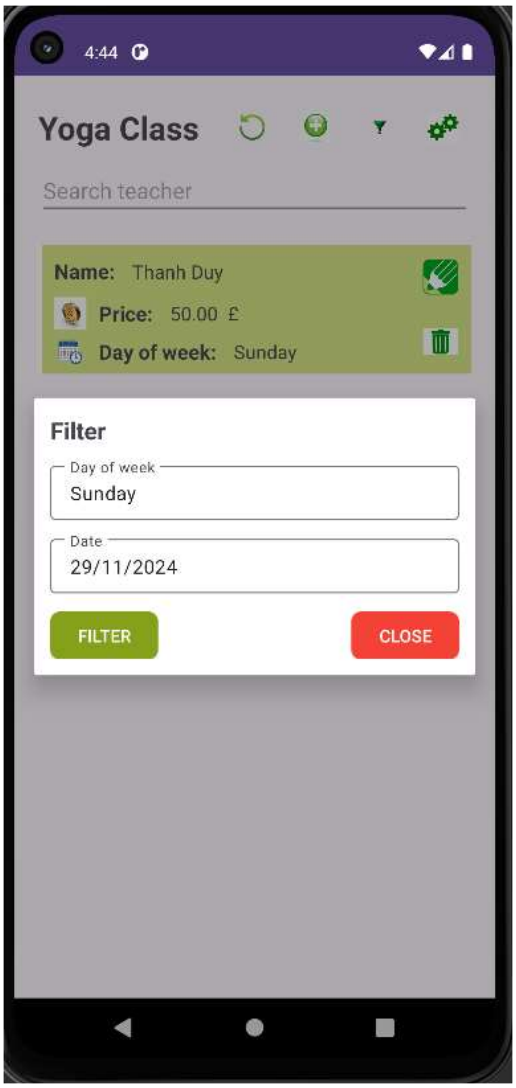

Figure 12 The filter functionality screen offers an intuitive interface that allows users to easily search for yoga classes based on specific criteria, enhancing usability and making the search process more efficient.

Search Functionality
Feature Demonstrated: Class Search
Description:
This screen allows users to search for specific yoga classes by criteria such as the teacher's name or other relevant details. The results are dynamically filtered and displayed based on the search query.
Functionalities:

- **Search Bar**: Users can enter a query (e.g., teacher's name) to filter the list of yoga classes.
- **Result Display**: Shows yoga classes that match the search criteria, including:
    - **Name**: Class name (e.g., "First Week Class").
    - **Price**: Cost per session in GBP (£).

- **Day of Week**: Day the class takes place.

- **Edit and Delete Options**: Users can edit (pencil icon) or delete (trash icon) individual classes directly from the search results.

Screenshot:

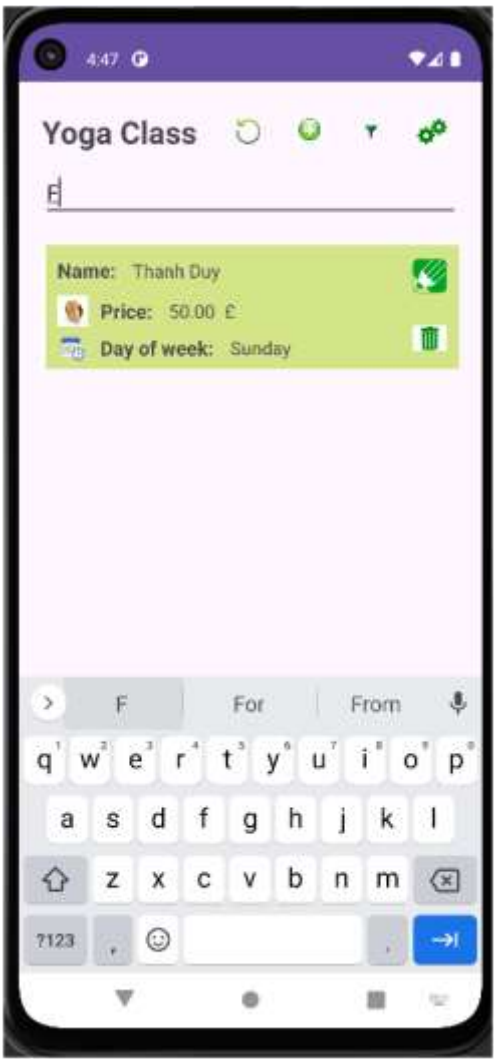

Figure 13 The search functionality screen illustrates how users can efficiently find yoga classes by filtering search results, enhancing navigation and overall user experience.

Class Instance Management
Feature Demonstrated: Managing Yoga Class Instances
Description:
This screen enables users to manage individual instances of yoga classes, allowing them to view, edit, or delete specific class occurrences for better scheduling control.
Functionalities:

- **Class Instance Details**: Displays information about a specific class session:
  - **Teacher**: Name of the instructor
  - **Date**: Date when the class instance is scheduled.
  - **Comment**: Optional notes or comments related to the class.

- **Edit and Delete Options**: Buttons for editing (pencil icon) or deleting (trash icon) a specific class instance.
- **Add New Instance**: A "+" button for adding new instances to the yoga class schedule.

Screenshot:

Figure 14 The class instance management screen provides detailed control over individual yoga sessions, allowing users to edit, delete, and add notes for each instance.

Add New Yoga Class

Feature Demonstrated: Add New Yoga Class Form
Description:
This screen allows users to input details for creating a new yoga class. It includes fields for entering the required information and provides validation to ensure all mandatory fields are completed.

Functionalities:

- **Input Fields**:
    - **Name**: Text input for the class name.

- **Day of Week**: Dropdown or input to select the day the class occurs.
- **Time of Course**: Input for the time the class is scheduled.
- **Capacity**: Input for the maximum number of participants.
- **Price**: Cost per session.
- **Duration**: Length of the class in minutes.
- **Type of Class**: Dropdown to select the type of yoga (e.g., Flow Yoga, Aerial Yoga).
- **Description**: Optional text input for additional details about the class.

- **Validation**: Fields are highlighted in red with error messages if mandatory fields are left empty.
- **Save Button**: Saves the entered details and adds the class to the database.

Screenshot:

Figure 15 The form for adding a new yoga class includes validation to ensure data accuracy and provides clear input fields for easy user interaction.

Network Connectivity Alert
Feature Demonstrated: Internet Connectivity Management
Description:
This screen emphasizes the need for a stable internet connection to ensure certain app features, such as syncing data to the cloud or retrieving real-time updates, work properly.

Functionalities:

- **Wi-Fi Toggle:** Allows users to turn Wi-Fi on or off.
- **Mobile Data Toggle:** Gives the option to enable or disable mobile data.
- **Status Display:** Shows the current connection status, such as "Wi-Fi is off" or "Mobile data not connected."
- **Done Button:** Lets users confirm their network settings and return to the app.

Usage in the App:
The app notifies users to check their internet connection when attempting actions that require cloud synchronization or real-time data access, ensuring seamless functionality.

Screenshot:

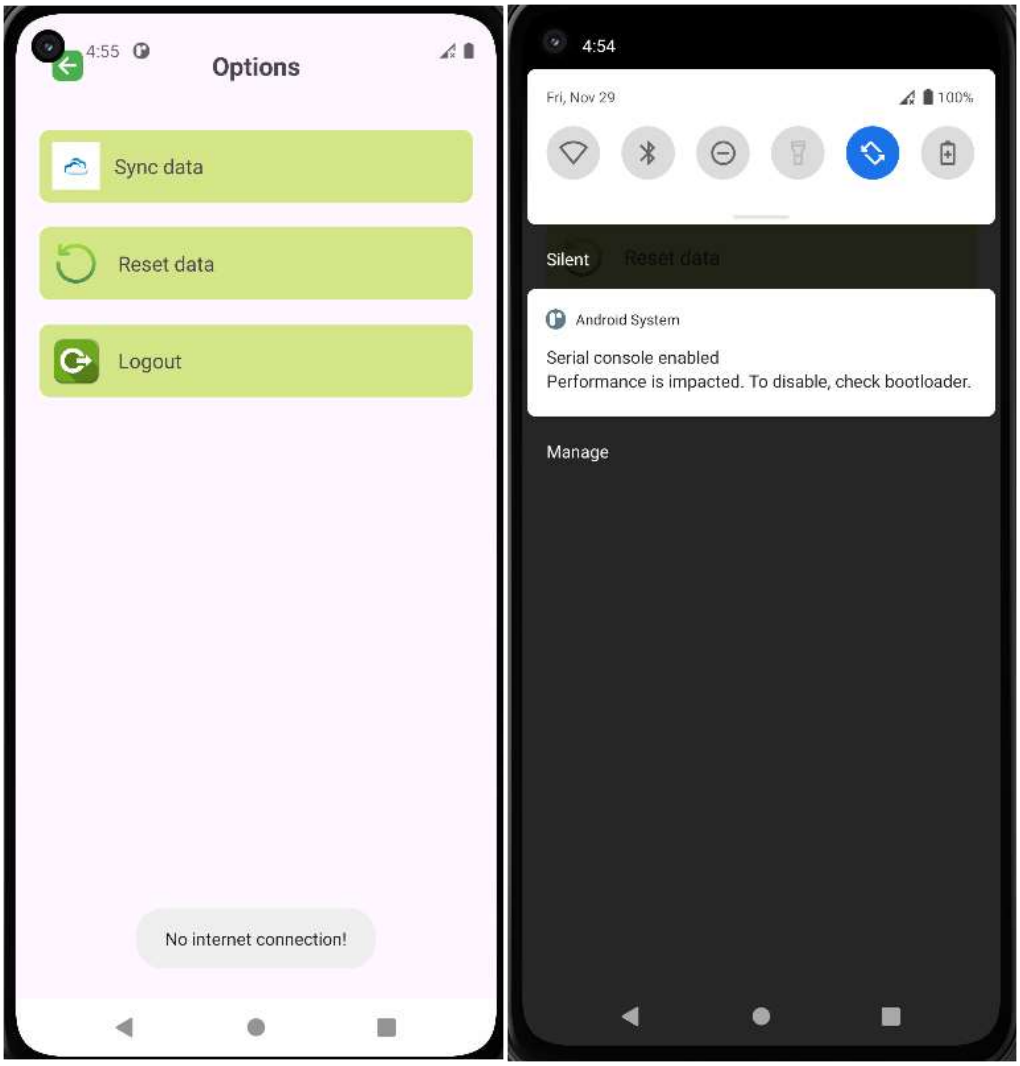

Figure 16 The network connectivity alert screen ensures that users are aware of their internet connection status and provides options to adjust settings for smooth app operation without disruptions.

The options menu screen offers access to essential app settings and administrative functions, such as syncing data, resetting the database, and logging out. It allows users to manage their app experience and data efficiently.

- Sync Data: Begins the process of uploading local data to the cloud (Firebase), ensuring updates are reflected across all devices.
- Reset Data: Clears the local database, reverting it to its default state. This feature is useful for testing or clearing app data.
- Logout: Logs the user out of the app, taking them back to the login screen.

Screenshot:

Figure 17 The options menu screen offers a streamlined, user-friendly interface that allows easy access to essential app features, ensuring smooth navigation and efficient management of user settings.

Create Class Instance
Feature Demonstrated: Adding a New Class Instance
Description:
This screen allows users to add a new instance of a yoga class by entering key details such as the date, teacher name, and optional comments. The form includes validation to ensure the correct and complete entry of data.

Functionalities:

- **Input Fields:**
  - **Date:** Users select the date for the class instance. If the entered date is invalid, an error message (e.g., "Invalid date") is displayed.
  - **Teacher:** A field where the name of the instructor is entered.
  - **Comment:** An optional text field where users can add additional notes or comments about the class instance.
- **Validation:**

- Ensures that the date is entered correctly, highlighting errors if the input is invalid.

- **Save Button:**
  - Located at the top-right corner of the screen, this button saves the entered class instance details to the database once all fields are correctly filled.

Screenshot:

Figure 18 The create class instance screen provides an easy-to-use form for adding specific class instances, with built-in validation to ensure data accuracy.

Hybrid Xamarin (Customer)
Main Screen

Feature Demonstrated: User Dashboard / Main Interaction Hub

Description:
This screen serves as the primary interface for customers, offering access to all core app features and services. The layout is designed to ensure smooth navigation, with easy access to key functionalities such as browsing products, placing orders, and managing personal settings.

- **Navigation Menu**: A side or bottom menu that provides access to essential sections, such as Home, Orders, Profile, and Settings.
- **Main Content Area**: Displays important information tailored to the user, including recent orders, available offers, or notifications.
- **Search Bar**: Enables users to quickly search for products or services from the home screen.
- **Action Buttons**: Easily accessible buttons that allow users to quickly place an order, view their cart, or manage their account.
- **Notifications/Alerts**: A dedicated section to display important updates, special offers, or messages for the user.

Screenshot:

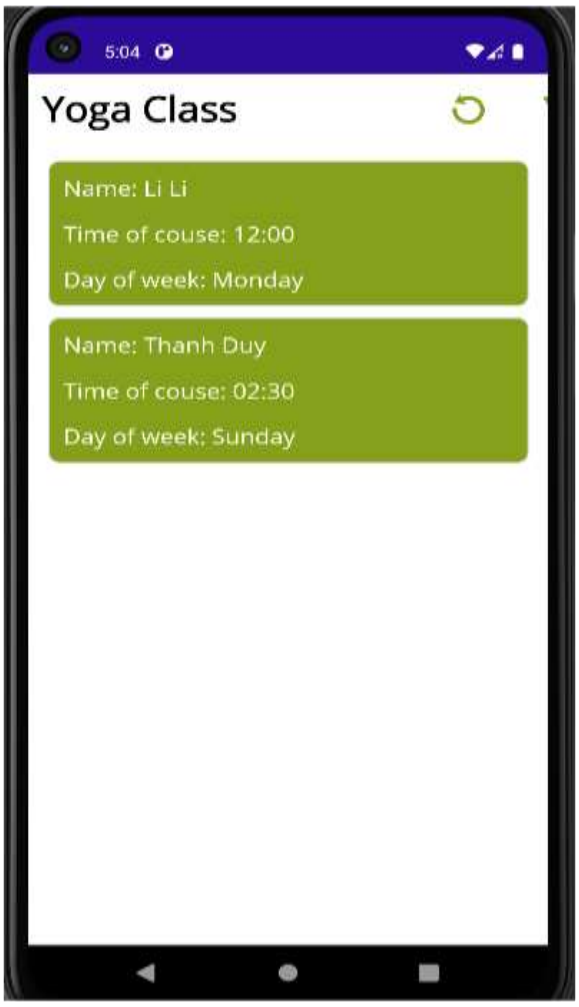

Figure 19 The main screen offers a well-organized layout with clear, easy-to-read text and vibrant buttons for essential actions. The user-friendly design prioritizes simplicity, allowing customers to quickly find the most relevant information. The dashboard high

The Yoga Class Detail Screen provides in-depth information about a specific class, making it easy for users to assess whether the class meets their needs.

Key features include:

- **Title**: Clearly labeled as "Yoga Class Detail."
- **Class Information**: Displays essential details such as the class name ("Yoga Beginner"), capacity (10 students), day and time (Monday at 18:30), duration (90 minutes), price (£90), class type (Flow Yoga), and a brief description for beginners.
- **Action Buttons**:
    - "View Class Instance List" allows users to check specific class session dates.
    - "Back to Yoga Classes" takes users back to the main class list for easy navigation.

This screen's simple design focuses on presenting vital information in a clear and easy-to-read manner, ensuring a smooth user experience.

Screenshot:

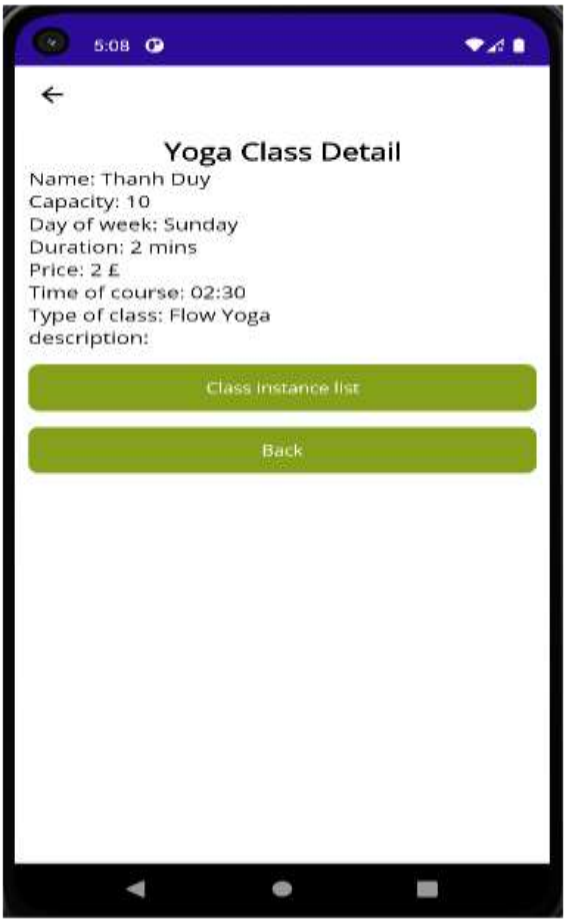

Figure 20 The screenshot showcases the detailed information screen for a yoga class, displaying essential details such as the class name, schedule, duration, pricing, and type. It provides a comprehensive overview, helping users make informed decisions before joi

Purpose of the Screen:

- **Provide Detailed Information:** This screen ensures that users can fully understand the details of a yoga class before making a booking, helping them make an informed choice.
- **Ease of Use:** The layout is simple and intuitive, making it easy for users to navigate and find important information about the class.
- **Aid Decision-Making:** By showcasing all necessary details, users can easily compare different classes and select the one that best fits their needs.

Evaluation: The screen's design is clean and the content is well-organized, making it easy to read. However, several improvements could further enhance the user experience:

- **Add Images:** Including relevant images could help users visually connect with the class and the type of yoga being offered.
- **Translate to Vietnamese:** Making the screen available in Vietnamese would increase accessibility for users who prefer or are more comfortable with the language.
- **Provide More Information:** Offering additional information, such as instructor profiles, dress code recommendations, or other helpful tips, could enrich the user's understanding and preparedness for the class.

Conclusion: This screen is effective in presenting key details about the yoga class. While it is functional, some enhancements like visual content, language options, and extra information could improve its accessibility and overall usability.

Class Instance List Screen

Feature Demonstrated: Yoga Class Session List

Description: This screen provides a list of specific sessions for a selected yoga class. It appears after a user selects a particular yoga class from the main class list. The goal is to help users select the most suitable session time based on availability and preferences.

- **Title:** "Class Instance" clearly indicates that this screen is dedicated to showing specific instances or sessions of a selected yoga class.
- **Session Information:**
    - **Teacher:** Displays the name of the instructor leading the session.
    - **Date:** Shows the scheduled date for each specific yoga session.
    - **Description:** Offers a brief overview of the session, which could include details like the theme or particular focus of the class (e.g., flexibility, relaxation).

Functionality:

- **Session Selection:** Users can tap on a specific session to view more details or register for that session.

- **Navigation Back:** The option to return to the main yoga class list screen, ensuring a smooth user flow.

This screen allows users to easily view all available instances of a yoga class and helps them make a decision based on the date, instructor, and the description of each session.

Screenshot:

Figure 21 The screenshot highlights the yoga class session list, displaying key information such as the instructor's name, session date, and a brief description of each class, enabling users to select the most suitable session based on their preferences.

Purpose of the Screen:

- **Display Yoga Class Options**: Provides a comprehensive list of available yoga classes for the user to browse through.
- **Advanced Search Functionality**: Allows users to filter the class list by their preferences (day of the week, time), making it easier to find suitable classes.

- **Easy Navigation**: The search and sorting features improve the user experience by providing quick access to relevant classes.

Evaluation: This screen effectively presents a list of yoga classes with a search option for filtering results based on day and time. However, there are a few opportunities for improvement:

- **Clearer Categorization**: Categorizing classes by difficulty level (e.g., beginner, intermediate, advanced) would help users make more informed choices.
- **Additional Filters**: More filter options such as class type (e.g., Vinyasa, Hatha) or class duration could provide users with a more tailored search experience.
- **Visual Enhancements**: Including icons or small images representing each class type might make the list more visually appealing and easier to navigate.

Comparison to the Previous Screen: Compared to the class instance list screen, which focuses on specific class sessions, this screen serves as an overview of all available classes. The main difference is that this screen offers users more flexibility to search and filter based on various criteria, making it easier to find classes that fit their schedule and preferences.

Conclusion: This screen successfully displays a list of available yoga classes and enhances the user experience by providing an advanced search feature. With some improvements in categorization, additional filter options, and visual enhancements, the usability and appeal of this screen could be further enhanced.

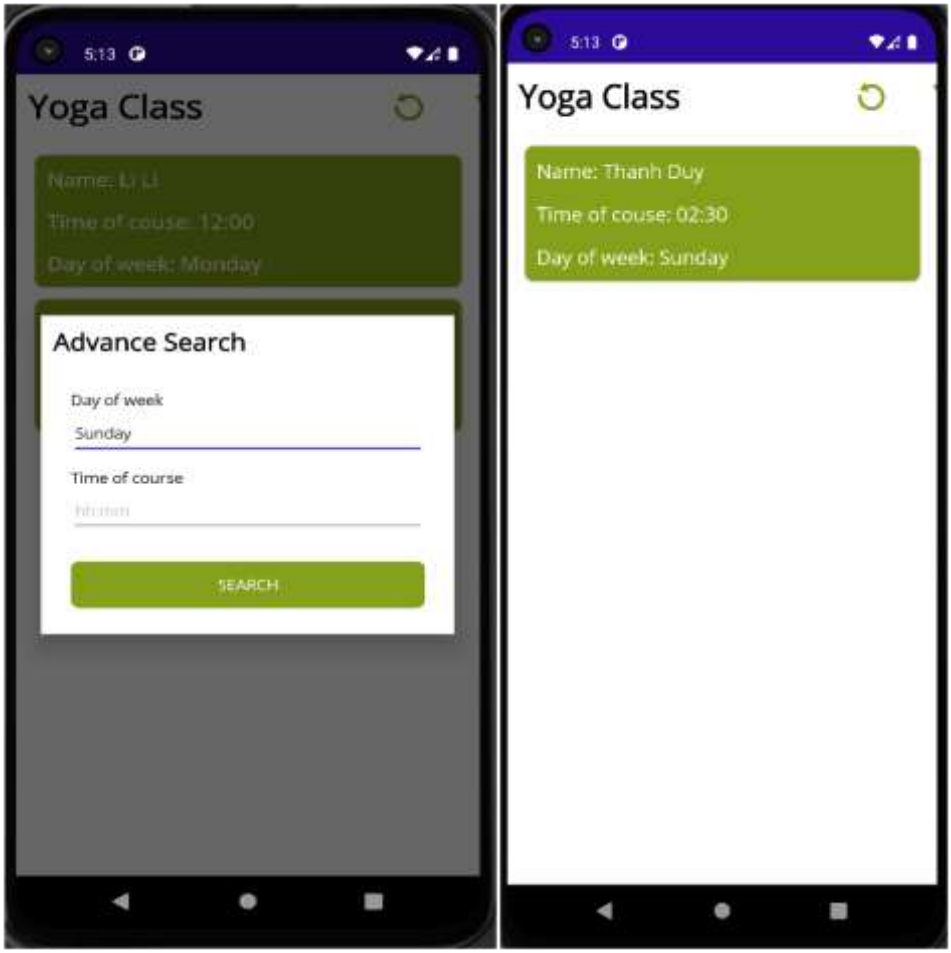

Figure 22 The screenshot demonstrates a list of available yoga classes, featuring an advanced search functionality that allows users to filter classes based on the day of the week and start time, making it easier to find classes that match their schedule and pref

Purpose of the Screen:

- **Display Yoga Class List:** Helps users easily browse through available yoga classes.
- **Search for Classes:** Enables users to filter and find classes based on their preferences, such as day and time.
- **User-Friendly Interface:** The design is simple, intuitive, and easy to navigate.

Evaluation:

- The screen's interface is simple and effective for browsing yoga classes. It allows users to easily find a class that fits their preferences. However, there are areas that could be improved:
    - **Detailed Class Information:** Providing more details, such as the instructor's name, room number, difficulty level, or a brief description of the class, would help users make more informed decisions.
    - **Additional Filters:** Additional filters, such as class type (e.g., Hatha, Vinyasa) or teacher, could help users narrow down their options further.

- **Booking Option:** Adding a "Book Now" or "Register" button next to each class would streamline the booking process and make it more convenient.

Comparison to the Previous Screen:

- **Class Detail Screen:** While the class detail screen focuses on providing in-depth information about a specific class, this screen displays a list of all available classes and allows users to filter their search. It offers an overview and enables users to choose the most suitable class based on their preferences.

Conclusion:

This screen effectively serves the purpose of displaying and searching through yoga classes. However, adding more detailed class information, extra filtering options, and a booking feature would enhance the user experience further.

Screenshot:

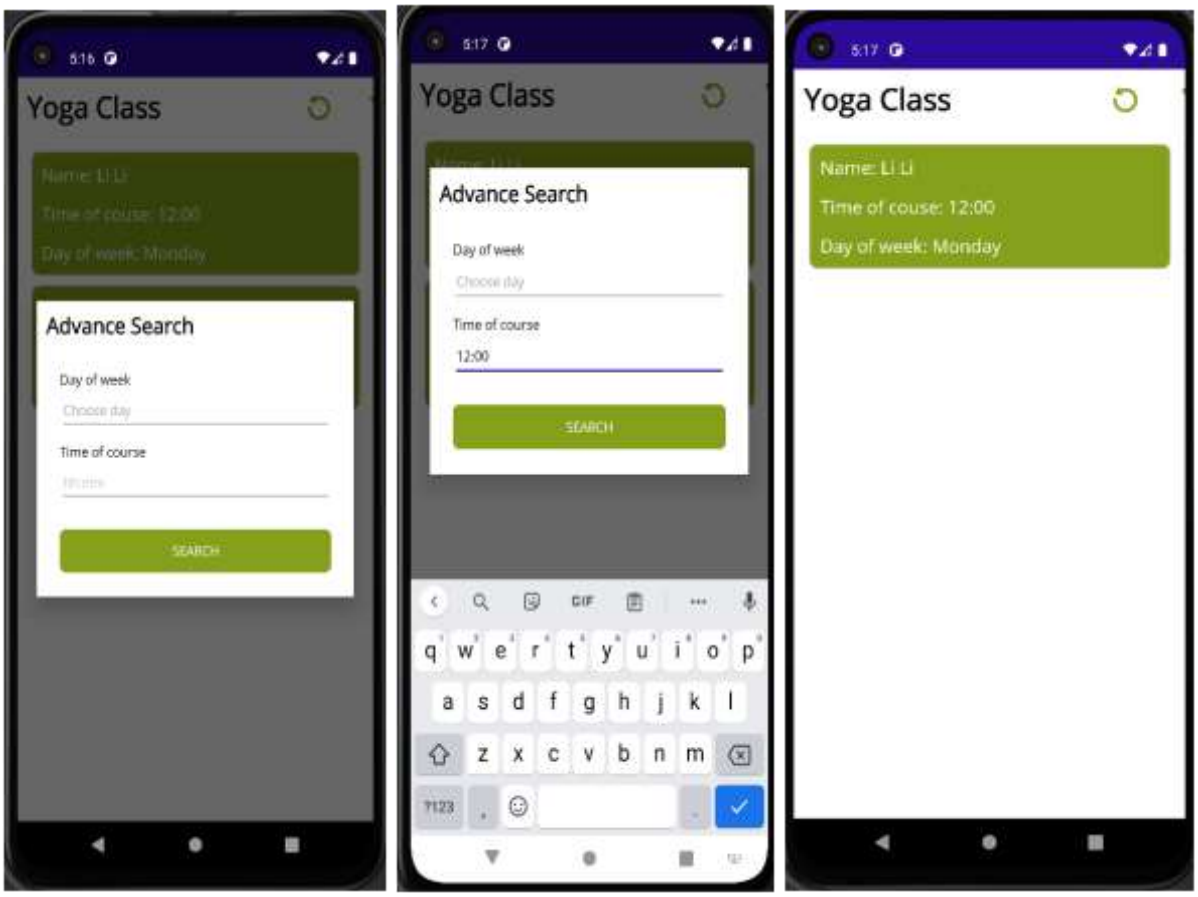

Figure 23 The screenshot shows the yoga class list with an advanced search feature, enabling users to filter classes by day and time for easier selection.

Purpose of the Screen:

- Display Class List: Allows users to easily browse through available yoga classes.

- Search for Classes: Enables users to quickly find classes by filtering based on day and time.
- User-Friendly Interface: Simple and intuitive design ensures a smooth user experience.

Evaluation and Suggestions for Improvement:

- **Design**: The current design is clear but could benefit from additional visual elements or colors to make it more engaging.
- **Advanced Search**:
    - **Day Selection**: Adding a dropdown list for day selection would simplify the search process.
    - **Additional Criteria**: Include filters for level (Beginner, Intermediate, Advanced), yoga type (e.g., Hatha, Vinyasa), and instructor.
- **Detailed Class Information**: Display additional class details like the instructor's name, class location, difficulty level, and a brief description of the session's focus.
- **Additional Features**:
    - **Booking**: Allow users to book classes directly through the app.
    - **Search History**: Enable users to save their search criteria for future use.
    - **Class Ratings**: Implement a rating system so users can evaluate classes after attending.

Conclusion: While the screen serves as a good starting point, adding more filtering options, detailed information, and features like booking and ratings would significantly improve the overall user experience.

VI. SECTION 5 – CODE.

Coursework (Admin)
Source File: **AdvanceSearchAlert.java**

The **AdvanceSearchAlert** class is a crucial feature in the application, specifically designed to provide an advanced search functionality through a custom dialog. This dialog allows users to refine their search by specifying details like dates and days of the week, enhancing the overall search experience.

Key Features:

1. **Custom Dialog Design**:
   The **AdvanceSearchAlert** class presents a clean and user-friendly custom dialog that focuses solely on the advanced search options. This helps keep the main interface clutter-free while providing a dedicated space for the search criteria.
2. **Date Selection**:
   One of the main features of the dialog is a date picker, which allows users to select

specific dates to filter their search. This is particularly useful for users who need to find results tied to a particular day.

3. **Day of the Week Filter**:
   The dialog also offers a filter for selecting the day of the week, allowing users to find events or sessions that occur on a specific day. This is useful for users with specific preferences for the days they want to attend.
4. **User Input Validation**:
   The class ensures that any user input is validated. If an invalid date is entered or no day is selected, an error message is displayed to help guide the user and prevent incorrect searches.
5. **Simple and Intuitive Interface**:
   The design is straightforward, with clearly labeled options for date and day selection. The layout ensures that users can easily interact with the dialog without confusion.
6. **Search Action**:
   Once the user has selected their desired search criteria, they can either confirm the selection by pressing "Apply" or cancel the operation. The confirmation action triggers the search process based on the selected parameters.

The **AdvanceSearchAlert** class plays a significant role in providing users with the tools to filter and narrow down their search criteria. Its clean design, intuitive user interface, and thoughtful validation make it an essential feature for improving the user experience. By offering advanced search options like date and day filters, this class contributes to making the app more efficient and user-centric.

A. Initialization of the Date Picker

The `initDateTimePicker` method initializes a custom date picker that allows users to select a specific date. It uses the `DateTimePicker` class to display a calendar with the day, month, and year format. When the user selects a date, the `txtDate` field is updated with the chosen date in the specified format, and the `Calendar` object is stored for further use. The date picker is seamlessly integrated into the app's UI and is triggered when the user clicks on the `txtDate` EditText field, providing real-time feedback.

```
private void initDateTimePicker() {
    dateTimePicker = new DateTimePicker(getSupportFragmentManager(),
DateTimePicker.D_M_Y);
    dateTimePicker.setOnPicked((calendar, dateTimeFormat) -> {
        bind.txtDate.getEditText().setText(dateTimeFormat);
        this.calendar = calendar;
    });
    bind.txtDate.getEditText().setOnClickListener(v -> {
        dateTimePicker.show();
    });
}
```

This function stands out because it uses a clean separation of concerns, with the DateTimePicker utility handling all the complexity of date management. By using the inline onDatePicked listener, it ensures a dynamic user experience, where the selected date is instantly reflected in the UI. Additionally, the integration with the UI binding

(bind.txtDate.getEditText()) makes the code more concise and eliminates the need for manual layout handling, improving code clarity and readability.

B. Initialization of the Day-of-Week Selector

The initDayOfWeekDialog function creates a dialog for selecting a day of the week. It loads day names from resources, uses an ArrayAdapter to display them, and when a user selects a day, it updates an input field with the day's name and stores its numeric value. This approach provides a user-friendly interface for selecting and processing days.

```
private void initDayOfWeekDialog() {
    String[] dayOfWeekData =
getResources().getStringArray(R.array.day_of_week);
    dayOfWeekAdapter = new ArrayAdapter<>(this,
android.R.layout.simple_list_item_1, new ArrayList<>());
    dayOfWeekAdapter.notifyDataSetChanged();
    for (int j = 0; j < dayOfWeekData.length; j++) {
        dayOfWeekAdapter.add(new UnitArrayAdapter<>(dayOfWeekData[j], j +
1));
    }

    dayOfWeekDialog = new AlertDialog.Builder(this)
            .setTitle("Day of week")
            .setAdapter(dayOfWeekAdapter, new
DialogInterface.OnClickListener() {
                @Override
                public void onClick(DialogInterface dialog, int which) {
                    dialog.dismiss();
                    UnitArrayAdapter<Integer> elementDayOfWeek =
dayOfWeekAdapter.getItem(which);

binding.txtDayOfWeek.getEditText().setText(elementDayOfWeek.getLabel());
                    dayOfWeek = elementDayOfWeek.getValue();
                }
            })
            .create();
}
```

I'm proud of this function because it efficiently integrates resource data and displays it in a clean, user-friendly manner using an `ArrayAdapter`. The logic behind updating the UI dynamically after a user selection is simple yet effective, ensuring an excellent user experience. Additionally, by storing both the name and numeric value of the day, the function ensures flexibility and reusability in the app.

C. Handling Search and Close Actions

The `initView` function initializes the UI components for the advanced search feature. It sets up the date picker and day of week dialog. It also defines the behavior for the "Search" button to capture the selected day and date, then triggers the `onSearch` method. For the "Close" button, it clears the selected date and day, resets the tag, and calls `onClose` to close the search alert.

```
private void initView() {
    InitDateTimePicker();
    InitDayOfWeekDialog();
    bind.btnSearch.setOnClickListener(new View.OnClickListener() {
```

```
        @Override
        public void onClick(View v) {
            int dayOfWeek = (int) bind.txtDayOfWeek.getEditText().getTag();
            String date = bind.txtDate.getEditText().getText().toString();
            listener.onSearch(dayOfWeek, date, AdvanceSearchAlert.this);
        }
    });

    bind.btnClose.setOnClickListener(new View.OnClickListener() {
        @Override
        public void onClick(View v) {
            bind.txtDate.getEditText().setText("");
            bind.txtDayOfWeek.getEditText().setText("");
            bind.txtDayOfWeek.getEditText().setTag(-1);
            listener.onClose(AdvanceSearchAlert.this);
        }
    });
```

I'm proud of this function because it efficiently sets up the user interface, linking buttons to their actions. It simplifies the process for users to search and reset their inputs, making the app easy to use and understand. The code is clean and maintains good functionality, improving the overall user experience.

Key Takeaways: This section of the code highlights my emphasis on:

- **User-Centric Design**: Ensuring a seamless and intuitive experience through elements like the date picker and day-of-week dialog, making it easy for users to interact with the app.
- **Code Readability**: Writing clear, modular functions that follow Android development best practices, which helps maintain the code and makes it easier for others to understand.
- **Robustness**: Implementing features that reset the state properly, reflect user inputs dynamically, and handle potential errors effectively to ensure a smooth app experience.

Source File: CreateYogaView.java

The CreateYogaView class handles the creation and editing of yoga class details. This key feature allows users to dynamically manage class information within the application. Below, I explain the most important and well-structured elements of this class, focusing on how it integrates user interactions with backend processes seamlessly.

A. Saving Yoga Class Data:

This method gathers all the form data, validates it, and creates or updates the YogaClass object. It then triggers the confirmation dialog to display the entered data.

```
private void saveYogaClass() {
    KeyboardUtil.HideKeyBoard(this);
    if (!CheckYogaClassData()) {
        return;
```

```
    }
    if (!isUpdate) {
        yogaClass = new YogaClass();
    }

    yogaClass.dayOfWeek = dayOfWeek;
    yogaClass.yogaName =
binding.txtName.getEditText().getText().toString();

yogaClass.setPrice(Double.parseDouble(binding.txtPrice.getEditText().getTex
t().toString()));

yogaClass.setDuration(Integer.parseInt(binding.txtDuration.getEditText().ge
tText().toString()));
    yogaClass.typeOfClass =
binding.txtTypeOfClass.getEditText().getText().toString();
    yogaClass.description =
binding.txtDes.getEditText().getText().toString();
    yogaClass.timeOfCourse =
binding.txtTimeOfCourse.getEditText().getText().toString();
    yogaClass.capacity =
Integer.parseInt(binding.txtCapacity.getEditText().getText().toString());
    displayConfirmSave(yogaClass);
}
```

B. Time Picker Dialog Setup:

This method initializes a time picker dialog where the user can select the course time. The time is formatted and displayed in the respective text field when set.

```
@SuppressLint("DefaultLocale")
private void initTimePickerDialog() {
    timePickerDialog = new TimePickerDialog(this, new
TimePickerDialog.OnTimeSetListener() {
        @Override
        public void onTimeSet(TimePicker view, int hourOfDay, int minute) {
            String time = String.format("%02d:%02d", hourOfDay, minute);
            binding.txtTimeOfCourse.getEditText().setText(time);
        }
    }, 0, 0, true);
```

C. Confirmation Dialog Before Saving:

This dialog displays the details of the yoga class before saving and asks for confirmation. If confirmed, the yoga class data is returned to the previous activity.

```
private void displayConfirmSave(YogaClass yogaClass) {
    new AlertDialog.Builder(this)
            .setTitle("Yoga Class information")
            .setMessage(yogaClass.toString())
            .setPositiveButton("ok", new DialogInterface.OnClickListener()
{
                @Override
                public void onClick(DialogInterface dialog, int which) {
                    dialog.dismiss();
                    Intent intent = new Intent();
                    intent.putExtra(YogaClass.class.getSimpleName(),
yogaClass);
```

```
                    int resultCode = isUpdate ? ViewResultCode.EDIT :
ViewResultCode.ADD;
                    setResult(resultCode, intent);
                    finish();
                }
            })
            .setNegativeButton("cancel", (dialog, which) -> {
                dialog.dismiss();
            }).show();
}
```

D. UI Binding & Initialization for Update:

This method binds the existing YogaClass data to the form fields if updating a yoga class, ensuring the UI reflects the current values before any changes are made.

```
private void bindUI() {
    binding.txtTypeOfClass.getEditText().setText(yogaClass.typeOfClass);
    binding.txtDes.getEditText().setText(yogaClass.description);
    binding.txtName.getEditText().setText(yogaClass.yogaName);

binding.txtCapacity.getEditText().setText(String.valueOf(yogaClass.capacity
));

binding.txtPrice.getEditText().setText(String.valueOf(yogaClass.getPrice())
);

binding.txtDayOfWeek.getEditText().setText(yogaClass.getDayOfWeekString());
    binding.txtTimeOfCourse.getEditText().setText(yogaClass.timeOfCourse);

binding.txtDuration.getEditText().setText(String.valueOf(yogaClass.getDurat
ion()));
}
```

Source File: `YogaDetailView`

YogaDetailView.java

The YogaDetailView class is an activity in the application that displays detailed information about a specific yoga class. It retrieves the YogaClass object from the intent, binds its properties (such as name, capacity, time, description, etc.) to the UI elements, and allows the user to navigate to another view to see class instances. It also includes a "Back" button to close the current view. The class utilizes ViewYogaDetailBinding for UI binding and ensures a clean, user-friendly interface for viewing yoga class details.

A. Binding data to the UI:

```
@SuppressLint("SetTextI18n")
private void bindUI() {
    YogaClass yogaClass = (YogaClass)
getIntent().getSerializableExtra(YogaClass.class.getSimpleName());
    bind.txtName.setText(yogaClass.yogaName);
    bind.txtMember.setText(yogaClass.capacity + "");
    bind.txtTimeOfCourse.setText(yogaClass.timeOfCourse);
    bind.txtDayOfWeek.setText(yogaClass.getDayOfWeekString());
    bind.txtTypeOfClass.setText(yogaClass.typeOfClass);
    bind.txtDuration.setText(yogaClass.getDurationString());
    bind.txtDes.setText(yogaClass.description);
```

```
    bind.txtPrice.setText(yogaClass.priceString());
    bind.btnViewClassInstance.setOnClickListener(new View.OnClickListener()
{
        @Override
        public void onClick(View v) {
            Intent intent = new Intent(YogaDetailView.this,
ClassInstanceView.class);
            intent.putExtra(YogaClass.class.getSimpleName(), yogaClass);
            startActivity(intent);
        }
    });
}
```

Retrieves the YogaClass object passed from the previous activity using getIntent().getSerializableExtra(). It then sets the values of various TextViews (like txtName, txtMember, etc.) using the properties of the YogaClass object.

B. Back button listener:

```
    bind.btnBack.setOnClickListener(v -> {
        finish();
    });
}
```

Sets an OnClickListener on the btnBack button. When clicked, it triggers the finish() method, which closes the current activity and returns to the previous one in the activity stack.

Source File: FirebaseService.java

The **FirebaseService** class plays a pivotal role in the application by managing the synchronization of data between the local database and Firebase Cloud Firestore. It guarantees that all yoga class and class instance information is consistently updated in the cloud. Below, I outline the key methods that demonstrate an effective implementation of cloud synchronization and data handling.

A. Syncing Yoga Classes and Class Instances

```
private CompletableFuture<Boolean> syncYogaThread(List<Map<String, Object>>
yogaClassHashList) {
    CompletableFuture<Boolean> thread = new CompletableFuture<>();
    FirebaseFirestore firebaseInstance = FirebaseFirestore.getInstance();
    firebaseInstance.collection(YogaClass.class.getSimpleName())
            .get().addOnCompleteListener(taskQuery -> {
                WriteBatch batchYogaClass = firebaseInstance.batch();
                for (QueryDocumentSnapshot document :
taskQuery.getResult()) {
                    batchYogaClass.delete(document.getReference());
                }
                for (Map<String, Object> yogaClassHash : yogaClassHashList)
{
                    DocumentReference docRef =
firebaseInstance.collection(YogaClass.class.getSimpleName()).document();
                    batchYogaClass.set(docRef, yogaClassHash);
                }
                batchYogaClass.commit()
                        .addOnSuccessListener(aVoid -> {
                            thread.complete(true);
```

```
                        })
                        .addOnFailureListener(e -> {
                            thread.complete(false);
                        });
            });

    return thread;
}
```

This method syncYogaThread synchronizes yoga class data with Firebase Firestore. It fetches the existing documents from Firestore and deletes them, then updates the database by adding the new yogaClassHashList data. A WriteBatch is used to perform multiple write operations (delete + add) in one transaction. If the operation succeeds, it completes the thread with true; otherwise, it completes with false.

B. Starting the Synchronization Process.

```
public void startSync() {
    CompletableFuture<Boolean> syncYogaClass =
syncYogaThread(convertYogaClassToHashList());
    CompletableFuture<Boolean> syncClassInstance =
syncClassInstanceThread(convertClassInstanceToHashList());
    CompletableFuture<Void> syncThreadAll =
CompletableFuture.allOf(syncYogaClass, syncClassInstance);
    syncThreadAll.thenRun(() -> {
        try {
            Boolean responseYogaClass = syncYogaClass.get();
            Boolean responseClassInstance = syncClassInstance.get();

            if (responseYogaClass && responseClassInstance) {
                event.success();
            } else {
                event.fail("sync to firebase failed");
            }
        } catch (Exception e) {
            Log.e("FirebaseService", "startSync: " + e.getMessage());
            event.fail("sync to firebase failed");
        }
```

The startSync method starts the synchronization of both yoga classes and class instances using CompletableFuture. It calls syncYogaThread and syncClassInstanceThread to perform the sync operations in parallel. After both sync tasks are finished, it checks the results. If both succeed, it calls event.success(); otherwise, it reports failure using event.fail().

C. Converting Yoga Class to Hash Map

```
private List<Map<String, Object>> convertYogaClassToHashList() {
    List<Map<String, Object>> yogaClassHashList = new ArrayList<>();
    for (int i = 0; i < yogaClasses.size(); i++) {
        Map<String, Object> yogaClassHash = new HashMap<>();
        YogaClass yogaClass = yogaClasses.get(i);
        yogaClassHash.put(YogaClassDB.NAME, yogaClass.yogaName);
        yogaClassHash.put(YogaClassDB.YOGA_CLASS_ID,
yogaClass.yogaClassID);
        yogaClassHash.put(YogaClassDB.PRICE, yogaClass.getPrice());
        yogaClassHash.put(YogaClassDB.CAPACITY, yogaClass.capacity);
        yogaClassHash.put(YogaClassDB.DAY_OF_WEEK, yogaClass.dayOfWeek);
```

```
        yogaClassHash.put(YogaClassDB.TYPE_OF_CLASS,
yogaClass.typeOfClass);
        yogaClassHash.put(YogaClassDB.DESCRIPTION, yogaClass.description);
        yogaClassHash.put(YogaClassDB.TIME_OF_COURSE,
yogaClass.timeOfCourse);
        yogaClassHash.put(YogaClassDB.DURATION, yogaClass.getDuration());
        yogaClassHashList.add(yogaClassHash);
    }
    return yogaClassHashList;
}
```

This method converts a list of YogaClass objects into a list of hash maps, where each hash map represents the properties of a YogaClass object. For each yoga class, the relevant fields are added to the hash map with corresponding keys. The method returns a list of these hash maps, which are used for Firebase synchronization.

- **Hybrid Xamarin (Customer)**
- **Programming Language**: C# (Xamarin)
  - **Source File: YogaClassViewModel.cs**

A. Navigation to Detail Page (OnItemClicked).

```
public async void OnItemClicked(YogaClass yogaClass)
 {

     YogaClassDetailPage yogaClassPage = new YogaClassDetailPage(yogaClass);
     await Application.Current.MainPage
          .Navigation
          .PushAsync(yogaClassPage, true);
 }
```

The method OnItemClicked handles navigation to a detailed view of the selected yoga class. This is a common design pattern in mobile apps, where clicking on a list item leads to a new page that shows more detailed information. The separation of the navigation logic in the ViewModel makes it easier to manage and test.

B. Search Functionality (SearchData):

```
        public void SearchData(int dayOfWeek, string timeOfCourse)
        {
            List<YogaClass> searchYogaList = new List<YogaClass>();
            foreach (YogaClass item in yogaClassList)
            {
                if (item.dayOfWeek == dayOfWeek || item.timeOfCourse ==
timeOfCourse)
                {
```

```
                searchYogaList.Add(item);
            }
        }

        this.yogaClassList.Clear();
        searchYogaList.ForEach(yogaClass =>
        {
            yogaClassList.Add(yogaClass);
        });
    }
```

The SearchData method allows filtering yoga classes based on certain criteria (e.g., day of the week or time of course). This feature enhances the app's usability by enabling users to quickly find specific classes from a potentially large list, without needing to reload the entire dataset.

C. **Asynchronous Data Fetching (LoadDataAsync)**:

```
public async void LoadDataAsync()
 {
     IsLoading = true;
     yogaClassList.Clear();
     List<YogaClass> yogaClasses = await firebaseService.GetYogaClass();
     yogaClasses.ForEach(yogaClass =>
     {
         yogaClassList.Add(yogaClass);
     });
     IsLoading = false;
 }
```

The method LoadDataAsync handles the fetching of yoga class data asynchronously from a service (firebaseService). Using async/await ensures that the app doesn't block the UI thread while waiting for data. This keeps the UI responsive, improving the overall user experience

- **Source File: ClassInstanceViewModel.cs**

A. ObservableCollection<ClassInstance> for Data Binding:

```
private ObservableCollection<ClassInstance> classInstanceList = new
ObservableCollection<ClassInstance>();
public ObservableCollection<ClassInstance> ClasseInstanceList
{
```

```
    get { return classInstanceList; }
    set { SetProperty(ref classInstanceList, value); }
}
```

ObservableCollection<T> is used for managing collections of data that need to be bound to the UI. In Xamarin's MVVM architecture, this collection is automatically updated in the UI whenever the underlying data changes.

B. **Constructor Accepting Class Instances**:

```
private List<ClassInstance> classInstances;
public ClassInstanceViewModel(List<ClassInstance> classInstances)
{
    this.classInstances = classInstances;
}
```

The constructor accepts a list of ClassInstance objects, which represents the data for the view model. This list is stored in the classInstances field.

C. Loading Class Instances (LoadClassInstance):

```
public async void LoadClassInstance()
 {
     this.classInstances.ForEach(classInstance =>
     {
         classInstanceList.Add(classInstance);
     });

 }
```

This method iterates over the list of ClassInstance objects and adds them to the classInstanceList (which is bound to the UI). While this method is currently synchronous, the async keyword is included, indicating potential future asynchronous operations (e.g., fetching data from a database or a network).

References

developer, 2024. *developer*. [Online] Available at: https://developer.android.com/topic/libraries/data-binding?hl=vi [Accessed 29 11 2024].

firebase, 2024. *firebase.* [Online] Available at: https://firebase.google.com/products/realtime-database [Accessed 29 11 2024].

Kanade, V., 2022. *spiceworks.* [Online] Available at: https://www.spiceworks.com/tech/artificial-intelligence/articles/what-is-hci/ [Accessed 29 11 2024].